Smart Video Highlight & GIF Generator

Aplicaţie locală de extragere automată a momentelor cheie din materiale video bazată pe analiză multimedia avansată

Autor: Petre Adrian, 331AB

Anul: 2026

Cuprins

1 Introducere **2**
1.1 Contextul şi motivarea alegerii temei 2
1.2 Obiectivele proiectului 2

2 Suportul Tehnic şi Fundamente Teoretice **3**
2.1 Analiza soluţiilor similare 3
2.2 Fundamentele analizei vizuale 3
2.2.1 Reprezentarea digitală a imaginii şi spaţiul de culoare HSV 3
2.2.2 Fluxul Optic Dens - Algoritmul Farnebäck 4
2.2.3 Detectarea schimbărilor de scenă prin histograme HSV 4
2.3 Fundamentele analizei audio 5
2.4 Compresia şi containerizarea video 5

3 Implementare şi Arhitectură **6**
3.1 Tehnologii şi biblioteci utilizate 6
3.2 Arhitectura aplicaţiei - Pipeline-ul de procesare 6
3.3 Controlul parametrilor de către utilizator 7
3.4 Logica de fuziune a scorurilor şi detectarea segmentelor 8
3.5 Optimizări de performanţă şi gestiunea memoriei 8
3.6 Crop vertical pentru social media 9

4 Utilizare şi prezentarea funcţionalităţii **9**
4.1 Instalare şi configurare 9
4.2 Interacţiunea cu utilizatorul 9
4.3 Rezultatele procesării 10

5 Concluzii **11**

1 Introducere

1.1 Contextul şi motivarea alegerii temei

În ultimii ani, consumul de conţinut video a suferit o transformare profundă. Platformele de socializare precum TikTok, Instagram Reels şi YouTube Shorts au impus formatul video scurt (sub 60 de secunde) ca standard dominant de comunicare vizuală. Conform raportului Cisco Annual Internet Report [7], traficul video va reprezenta peste 82% din totalul traficului de internet, iar formatele scurte, verticale, au devenit principala metodă de creştere organică pentru creatorii de conţinut.

Editorii de conţinut pentru reţelele sociale se confruntă zilnic cu o provocare semnificativă: extragerea momentelor cele mai intense din materiale video de lungă durată. Un editor care realizează compilaţii din meciuri de fotbal trebuie să vizioneze integral partida de 90 de minute pentru a identifica fazele spectaculoase. Similar, un creator care produce „edits" din scene de film sau fragmente de podcast petrece ore întregi vizionând materialul brut, selectând manual secvenţele de climax vizual şi auditiv, şi decupându-le una câte una.

Acest proces manual prezintă trei dezavantaje majore: consumă foarte mult timp, este subiectiv (depinde de atenţia şi experienţa editorului) şi este repetitiv - aceleaşi operaţiuni de vizionare-selectare-decupare se repetă pentru fiecare proiect. Situaţia devine şi mai problematică atunci când editorul lucrează pe un calculator personal cu resurse limitate (de exemplu, un laptop cu doar 8 GB de memorie RAM), care nu poate rula instrumente profesionale grele precum Adobe Premiere Pro sau DaVinci Resolve fără compromisuri severe de performanţă.

Pornind de la aceste observaţii, prezentul proiect propune o soluţie: o aplicaţie web locală, inteligentă, care automatizează procesul de preselectare a momentelor de interes maxim dintr-un videoclip. Aplicaţia analizează sincronizat atât componenta vizuală (mişcare, schimbări de scenă), cât şi pe cea auditivă (volum, spectru, tranziţii), generând automat un montaj final adaptat nativ pentru platformele de social media.

Ceea ce diferenţiază fundamental această aplicaţie de soluţiile existente pe piaţă este faptul că procesarea se realizează integral pe calculatorul utilizatorului, fără a necesita conexiune la internet, fără a trimite datele către servere externe, fără GPU dedicat şi fără costuri recurente de abonament.

1.2 Obiectivele proiectului

Obiectivul principal al proiectului este realizarea unei aplicaţii web locale capabile să analizeze sincronizat fluxul vizual şi cel audio al unui clip video, identificând automat segmentele de interes maxim şi generând un fişier final ce conţine doar aceste momente.

Obiectivele secundare sunt:

- **Procesare 100% locală**, fără dependenţă de API-uri cloud sau servicii externe cu plată.
- **Funcţionare fără GPU dedicat**, utilizând algoritmi deterministici de procesare de semnal în locul modelelor masive de Deep Learning.
- **Compatibilitate cu hardware modest** - aplicaţia a fost proiectată şi testată pe un laptop cu doar 8 GB de RAM, demonstrând că rezultate de calitate pot fi obţinute şi pe echipamente

accesibile.

- **Control granular din partea utilizatorului** - posibilitatea de a ajusta în timp real ponderea componentei vizuale faţă de cea audio, precum şi pragul de selectivitate al algoritmului, direct din interfaţa aplicaţiei.
- **Export configurabil**, adaptat pentru social media: format landscape 16:9 (trailere), format vertical 9:16 (TikTok / Instagram Reels) sau animaţie WebP.

2 Suportul Tehnic şi Fundamente Teoretice

2.1 Analiza soluţiilor similare

Înainte de a proiecta propria soluţie, am analizat principalele instrumente disponibile pe piaţă care adresează o problematică similară.

Adobe Premiere Pro - Auto Reframe [8] reprezintă funcţionalitatea integrată în suita Adobe care permite reformatarea automată a unui clip pentru diferite raporturi de aspect. Punctele forte constau în calitatea profesională a exportului şi integrarea nativă cu ecosistemul Adobe. Punctele slabe sunt însă semnificative: costul ridicat al abonamentului lunar (peste 20 EUR/lună), necesitatea unui GPU dedicat pentru performanţă acceptabilă, şi faptul că Auto Reframe *nu* realizează detecţie de conţinut bazată pe analiza auditivă - reformatează cadrul, dar nu identifică momentele de interes.

OpusClip [9] este o platformă online care utilizează inteligenţă artificială generativă pentru a transforma videoclipuri lungi în clipuri scurte cu subtitrări automate. Punctul forte este calitatea subtitrărilor generate. Punctele slabe constau în necesitatea obligatorie de a încărca fişierul pe serverele companiei (probleme de confidenţialitate şi latenţă pentru fişiere mari), modelul freemium cu limite stricte de procesare şi imposibilitatea de a funcţiona offline.

CapCut Auto-Edit [10] oferă funcţionalităţi de editare automată, fiind integrat cu platforma TikTok (ambele aparţin ByteDance). Deşi este gratuit, CapCut nu permite controlul granular al parametrilor de detecţie şi depinde de conexiunea la internet pentru funcţionalităţile avansate.

Gling [11] este specializat în eliminarea momentelor de tăcere din podcasturi şi vlog-uri. Deşi performant în domeniul său, Gling este focalizat exclusiv pe analiza vocală şi nu detectează acţiunea vizuală, fiind inadecvat pentru editarea meciurilor de fotbal sau a scenelor de film.

În concluzie, niciuna dintre soluţiile analizate nu oferă **simultan**: procesare offline, control granular al ponderilor vizual/audio direct din interfaţă, funcţionare pe hardware modest (8 GB RAM, fără GPU) şi export nativ pentru social media. Proiectul de faţă adresează exact această nişă.

2.2 Fundamentele analizei vizuale

2.2.1 Reprezentarea digitală a imaginii şi spaţiul de culoare HSV

O imagine digitală este reprezentată intern ca o matrice tridimensională de valori numerice. Pentru o imagine color cu rezoluţia $W \times H$ pixeli şi 3 canale de culoare (BGR în convenţia OpenCV [1]), reprezentarea ocupă $W \times H \times 3$ octeţi în memorie. Spaţiul de culoare RGB codifică fiecare

pixel prin intensităţile canalelor roşu, verde şi albastru. Cu toate acestea, pentru analiza schimbărilor vizuale, spaţiul HSV (Hue - nuanţă, Saturation - saturaţie, Value - luminozitate) prezintă un avantaj semnificativ: decuplează informaţia cromatică de cea de iluminare [1]. Astfel, o variaţie de lumină (de exemplu, trecerea unui nor) nu este interpretată eronat ca o schimbare de scenă.

2.2.2 Fluxul Optic Dens - Algoritmul Farnebäck

Pentru a cuantifica intensitatea mişcării dintr-un videoclip fără a antrena o reţea neuronală, proiectul utilizează **algoritmul de flux optic dens** propus de Gunnar Farnebäck în 2003 [2]. Spre deosebire de fluxul optic rar (care urmăreşte doar anumite puncte de interes), fluxul dens calculează vectorul de deplasare pentru *fiecare pixel* din imagine, oferind o hartă completă a mişcării.

Algoritmul aproximează vecinătatea fiecărui pixel folosind **expansiuni polinomiale de gradul doi**. Fie $f_1(x)$ şi $f_2(x)$ reprezentările polinomiale ale aceluiaşi pixel în două cadre consecutive. Deplasarea d este estimată minimizând diferenţa:

$$f_2(x) \approx f_1(x-d) \tag{1}$$

Magnitudinea fluxului optic pentru un pixel la coordonatele (x,y) este:

$$M(x,y) = \sqrt{dx^2 + dy^2} \tag{2}$$

Scorul de mişcare al unui cadru întreg este calculat ca media aritmetică a magnitudinilor tuturor pixelilor, normalizată la diagonala cadrului pentru a fi independentă de rezoluţie:

$$S_{flow} = \frac{1}{W \cdot H} \sum_{x,y} M(x,y) \Big/ \sqrt{W^2 + H^2} \tag{3}$$

Implementarea utilizează o piramidă de imagine pe 3 niveluri cu o fereastră de analiză de 15×15 pixeli şi 3 iteraţii de rafinare, parametri care asigură un echilibru optim între precizie şi viteză de calcul.

2.2.3 Detectarea schimbărilor de scenă prin histograme HSV

Complementar fluxului optic (care detectează mişcarea *în cadrul* unei scene), s-a implementat un detector de **schimbări de scenă** bazat pe compararea histogramelor de culoare. Histograma 2D Hue-Saturation (cu 50 bin-uri pe canalul H şi 60 pe canalul S) oferă o amprentă compactă a distribuţiei cromatice a fiecărui cadru.

Compararea între două cadre consecutive se realizează prin **corelaţia Pearson**:

$$r = \frac{\sum_i (H_1(i) - \bar{H_1})(H_2(i) - \bar{H_2})}{\sqrt{\sum_i (H_1(i) - \bar{H_1})^2 \cdot \sum_i (H_2(i) - \bar{H_2})^2}} \tag{4}$$

Scorul de schimbare de scenă este definit ca $S_{scene} = \max(0, 1-r)$. O valoare apropiată de 0 indică cadre vizual similare, în timp ce o valoare apropiată de 1 semnalizează o tranziţie bruscă (tăietură de montaj, explozie, schimbare dramatică de cadru).

Scorul vizual final combină cele două semnale printr-o medie ponderată:

$$S_{vizual}(t) = 0{,}7 \cdot S_{flow}(t) + 0{,}3 \cdot S_{scene}(t) \tag{5}$$

2.3 Fundamentele analizei audio

Un moment intens vizual este de cele mai multe ori susţinut de o componentă auditivă puternică: strigătul comentatorului la un gol, o explozie într-o scenă de film, un „beat drop" într-un podcast muzical. Aplicaţia calculează trei descriptori spectrali complementari folosind biblioteca *librosa* [4]:

1. Energia RMS (Root Mean Square) măsoară anvelopa de volum sonor pe ferestre scurte de timp (2048 de eşantioane, cu un pas de 512). Formula de calcul este:

$$RMS = \sqrt{\frac{1}{N}\sum_{n=1}^{N} x[n]^2} \tag{6}$$

unde $x[n]$ sunt eşantioanele audio din fereastra curentă. RMS este proporţional cu puterea semnalului şi captează momentele de volum maxim.

2. Centroidul Spectral reprezintă „centrul de masă" al spectrului de frecvenţă şi se calculează:

$$SC = \frac{\sum_{k=1}^{K} f_k \cdot |S(k)|}{\sum_{k=1}^{K} |S(k)|} \tag{7}$$

unde f_k este frecvenţa bin-ului k, iar $|S(k)|$ este magnitudinea spectrală. Sunetele strălucitoare (aplauze, ţipete) au un centroid spectral ridicat, în timp ce sunetele grave, înfundate, au un centroid scăzut. Acest descriptor ajută la diferenţierea momentelor de climax acustic.

3. Onset Strength (Fluxul Spectral) măsoară energia tranzitorie - variaţiile bruşte ale spectrului de la o fereastră la alta. Este ideal pentru detectarea „beat drops", loviturilor percutante sau exploziilor sonore.

Scorul audio final fuzionează cei trei descriptori, fiecare normalizat în intervalul $[0,1]$ prin scalare min-max [6]:

$$S_{audio}(t) = 0{,}5 \cdot RMS_{norm}(t) + 0{,}2 \cdot SC_{norm}(t) + 0{,}3 \cdot Onset_{norm}(t) \tag{8}$$

2.4 Compresia şi containerizarea video

Videoclipurile procesate de aplicaţie sunt stocate în containere multimedia standardizate. Formatul de export principal este **MP4** cu codec H.264 (MPEG-4 AVC) [3], care utilizează compresia inter-cadru bazată pe predicţie de mişcare şi transformata cosinus discretă (DCT) pentru a reduce drastic dimensiunea fişierului menţinând calitatea vizuală. Alternativ, aplicaţia suportă exportul în format **WebM** cu codec VP8, un format deschis optimizat pentru distribuţia web. Transcodarea este realizată de FFmpeg prin intermediul bibliotecii MoviePy [5].

3 Implementare şi Arhitectură

3.1 Tehnologii şi biblioteci utilizate

Proiectul este dezvoltat integral în **Python 3** şi utilizează un ecosistem de biblioteci open-source:

Tabela 1: Tehnologiile utilizate în proiect

Componentă	**Bibliotecă**	**Rol**
Interfaţă web	Streamlit [12]	Încărcarea fişierelor, slidere de configurare, previzualizare video
Citirea cadrelor	OpenCV [1]	Decodarea containerelor video, calcul flux optic, histograme HSV
Export video	MoviePy + FFmpeg [5]	Tăierea subclipurilor, concatenare, transcodare H.264/VP8
Analiză audio	librosa [4]	Extragerea descriptorilor spectrali: RMS, centroid, onset
Procesare numerică	NumPy	Operaţiuni pe matrice, normalizare, interpolare
Detectare vârfuri	SciPy	Funcţia `find_peaks` pentru localizarea maximelor locale
Vizualizare	Matplotlib	Graficele scorurilor pe axa timpului

3.2 Arhitectura aplicaţiei - Pipeline-ul de procesare

Aplicaţia urmează o arhitectură de tip **pipeline liniar paralelizat**, în care fiecare etapă primeşte datele de la etapa anterioară şi produce un rezultat intermediar. Etapele vizuală şi audio rulează **în paralel** pe thread-uri separate.

Fluxul complet de procesare este următorul:

1. **Încărcare şi extragere metadate:** Se extrag rezoluţia, FPS-ul nativ, durata şi numărul total de cadre fără a decoda pixelii.
2. **Eşantionare cadre (Sampling):** Fluxul video este eşantionat la o rată redusă configurabilă (implicit 10 cadre/secundă) şi redimensionat la 360p pentru analiză.
3. **Extragere audio:** Pista audio este extrasă ca fişier WAV mono la 22 050 Hz.
4. **Calcul scoruri vizuale:** Se calculează scorul de flux optic şi scorul de schimbare de scenă.
5. **Calcul scoruri audio:** Din fişierul WAV se extrag RMS, centroidul spectral şi onset strength.
6. **Fuziune şi detecţie:** Scorurile sunt normalizate, interpolate pe o axă temporală comună şi combinate conform ponderilor stabilite de utilizator.
7. **Export:** Segmentele identificate sunt decupate, concatenate cu tranziţii fade şi exportate.

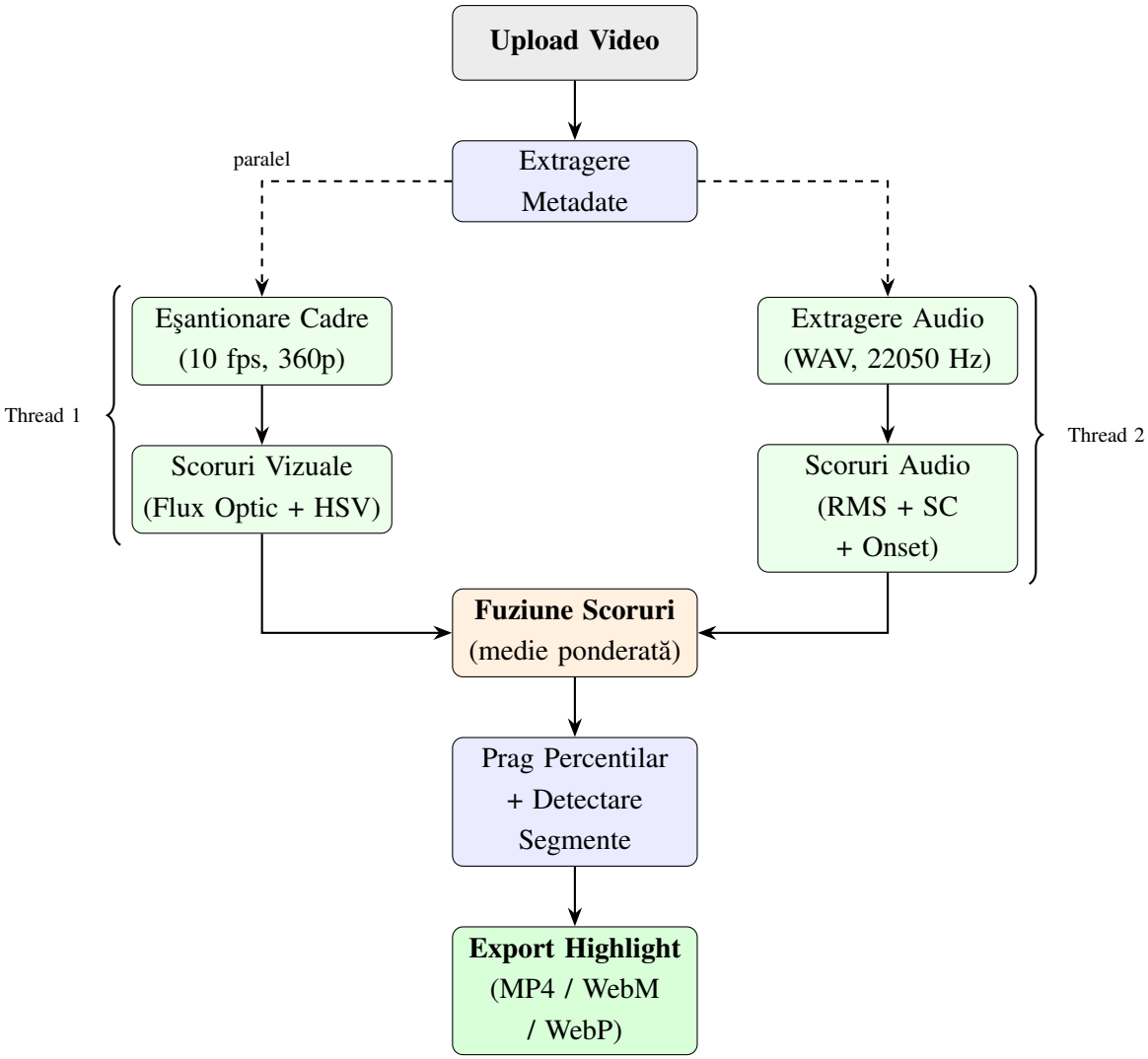


Figura 1: Arhitectura pipeline-ului de procesare a aplicaţiei

3.3 Controlul parametrilor de către utilizator

Un aspect esenţial al aplicaţiei îl constituie posibilitatea utilizatorului de a **configura în timp real** comportamentul algoritmului de detecţie, direct din interfaţa grafică, fără a necesita cunoştinţe de programare.

Ponderile componentelor vizuală şi audio sunt controlate prin două slidere independente. Utilizatorul poate decide cât de mult contează mişcarea vizuală şi cât de mult contează energia sonoră în decizia finală de selecţie. De exemplu, pentru un meci de fotbal unde comentatorul strigă la fiecare fază importantă, se recomandă ponderi echilibrate (0.5/0.5). Pentru o compilaţie de scene de acţiune dintr-un film mut, utilizatorul poate seta ponderea vizuală la 0.9 şi pe cea audio la 0.1. Aceste ponderi sunt normalizate intern astfel încât suma lor să fie întotdeauna egală cu 1:

$$F(t) = \frac{w_v}{w_v + w_a} \cdot S_{vizual}(t) + \frac{w_a}{w_v + w_a} \cdot S_{audio}(t) \tag{9}$$

Pragul (Threshold Percentile) permite utilizatorului să stabilească nivelul de selectivitate. O valoare de 75 înseamnă că doar momentele al căror scor fuzionat se situează în primele 25% ca

intensitate vor fi incluse în montajul final. O valoare de 90 selectează doar primele 10%, producând un montaj mai scurt dar cu momente şi mai intense. Acest mecanism adaptiv este superior unui prag fix deoarece se calibrează automat la distribuţia scorurilor fiecărui videoclip în parte.

De asemenea, utilizatorul controlează **durata minimă a unui clip** (pentru a evita fragmente prea scurte), **durata maximă totală** a montajului final (bugetul de timp), şi **presetul de export** (Trailer 16:9, TikTok/Reel 9:16, sau Animaţie WebP).

3.4 Logica de fuziune a scorurilor şi detectarea segmentelor

După calculul independent al scorurilor vizuale şi audio, acestea trebuie combinate pe o axă temporală unică. Deoarece cele două serii temporale pot avea rate de eşantionare diferite, se realizează o **interpolare liniară** pe o grilă temporală comună la 10 Hz.

Scorul fuzionat $F(t)$ este supus unui **prag procentual**. Momentele în care $F(t)$ depăşeşte pragul formează segmente contigue. Algoritmul detectează frontierele prin identificarea *rising edges* şi *falling edges* pe masca booleană $F(t) \geq T$.

Fiecărui segment detectat i se aplică un **peak padding** de 1,5 secunde pe fiecare parte. Această izolaţie temporală simetrică asigură captarea contextului - de exemplu, la un gol de fotbal, algoritmul nu decupează doar secunda exactă a golului, ci prinde şi construcţia fazei şi celebrarea de la final. Segmentele care se suprapun sunt **fuzionate automat**. Dacă durata totală depăşeşte bugetul stabilit de utilizator, se aplică o **selecţie greedy**: segmentele sunt sortate descrescător după scorul mediu şi sunt incluse până la epuizarea bugetului.

3.5 Optimizări de performanţă şi gestiunea memoriei

Contribuţia tehnică majoră a acestui proiect o constituie implementarea unor optimizări care permit funcţionarea aplicaţiei pe hardware modest. Proiectul a fost dezvoltat şi testat pe un **laptop cu 8 GB de memorie RAM**, o configuraţie frecvent întâlnită în rândul studenţilor şi al creatorilor de conţinut la început de drum. Această constrângere a impus soluţii inginerești concrete:

1. Procesare pe calupuri (Chunked Processing): Videoclipurile HD sau 4K decomprimate în matrice NumPy pot epuiza rapid 8 GB de RAM. Pentru a preveni erorile de tip *Out of Memory*, cadrele nu sunt stocate global. Modulul `video_processor` utilizează o funcţie generator (instrucţiunea Python `yield`) care produce calupuri de câte 500 de cadre. Fiecare calup este procesat, după care matricele masive sunt eliberate din memorie prin Garbage Collection. Această abordare de tip *stream processing* limitează consumul de memorie la un nivel constant, independent de lungimea videoclipului.

2. Redimensionare la analiză: Cadrele sunt redimensionate la o înălţime maximă de 360 de pixeli înainte de analiza vizuală. Calitatea detecţiei nu este afectată semnificativ, dar consumul de memorie se reduce de aproximativ 9 ori pentru un videoclip Full HD (1080p).

3. Paralelism: Procesarea vizuală şi cea audio rulează concurent pe thread-uri separate folosind `concurrent.futures.ThreadPoolExecutor`, reducând timpul total la durata celei mai lente operaţiuni.

4. Tranziţii pe 8 biţi: Efectele de fade-in şi fade-out sunt implementate printr-un filtru personalizat care operează pe matrice de tip `uint8`, nu pe `float64` (cum face MoviePy implicit). Aceasta reduce consumul de memorie la export cu un factor de 8.

5. Limitarea rezoluţiei la export: Videoclipul final este limitat la maximum 720p, asigurând un consum echilibrat de memorie în faza de randare.

3.6 Crop vertical pentru social media

Pentru presetul TikTok/Reel, aplicaţia realizează o **tăiere centrată** a cadrului la raportul de aspect 9:16. Algoritmul calculează noua lăţime $W' = H \times \frac{9}{16}$, o ajustează la cel mai apropiat număr par (cerinţă a codecurilor video), apoi extrage banda centrală a cadrului.

4 Utilizare şi prezentarea funcţionalităţii

4.1 Instalare şi configurare

Aplicaţia necesită **Python 3.9+** şi **FFmpeg** instalat în PATH-ul sistemului. Paşii de instalare sunt:

1. Crearea unui mediu virtual: `python -m venv venv`
2. Activarea mediului: `venv\Scripts\activate` (Windows)
3. Instalarea dependenţelor: `pip install -r requirements.txt`
4. Lansarea aplicaţiei: `streamlit run app.py`

Aplicaţia se deschide automat în browser-ul implicit la adresa `http://localhost:8501`. Toate operaţiunile se desfăşoară local.

4.2 Interacţiunea cu utilizatorul

Interfaţa a fost proiectată cu scopul de a fi accesibilă atât utilizatorilor tehnici, cât şi celor fără experienţă în editare video. Zona principală permite încărcarea fişierului video prin drag-and-drop (formate acceptate: MP4, AVI, MOV, MKV, WebM). Imediat după încărcare, aplicaţia afişează metadatele videoclipului: durata, rezoluţia, FPS-ul nativ şi numărul total de cadre.

Bara laterală (sidebar) conţine toate controalele de configurare, organizate în trei grupuri:

Ponderile scorurilor - două slidere care permit setarea importanţei relative a componentei vizuale şi audio. Această funcţionalitate este deosebit de utilă deoarece diferite tipuri de conţinut necesită ponderi diferite:

- *Meciuri de fotbal*: ponderi echilibrate (0.5 vizual, 0.5 audio) - atât acţiunea de pe teren cât şi strigătul comentatorului sunt relevante.
- *Scene din filme de acţiune*: ponderea vizuală mai mare (0.7/0.3) - mişcarea pe ecran este indicatorul principal.
- *Podcasturi*: ponderea audio mai mare (0.3/0.7) - variaţiile de voce şi ritmul conversaţiei contează mai mult decât imaginea statică.

Parametrii de detecţie - pragul (controlul selectivităţii), durata minimă per clip şi durata maximă totală a montajului.

Setările de export - presetul de proiect cu trei opţiuni: Standard Trailer (16:9 landscape), TikTok/Reel (9:16 vertical cu crop centrat), sau WebP Animation.

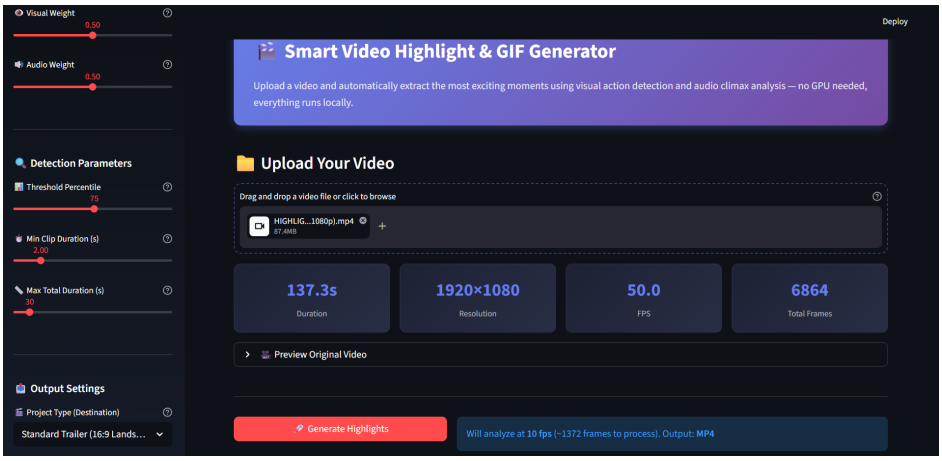


Figura 2: Interfaţa principală: zona de încărcare, metadatele video şi parametrii de configurare

4.3 Rezultatele procesării

Odată finalizată analiza, aplicaţia prezintă utilizatorului trei tipuri de rezultate:

1. Graficul cronologic al scorurilor (Score Timeline) - un grafic Matplotlib care reprezintă vizual evoluţia celor trei scoruri pe axa timpului. Linia turcoaz indică scorul vizual (mişcare + schimbări de scenă), linia roşie indică scorul audio (volum + spectru + tranziţii), iar linia galbenă îngroşată reprezintă scorul fuzionat. Linia orizontală punctată trasează nivelul pragului, iar zonele umbrite vertical marchează exact segmentele selectate. Acest grafic constituie dovada transparentă a modului de funcţionare al algoritmului - utilizatorul poate înţelege *de ce* au fost alese anumite momente.

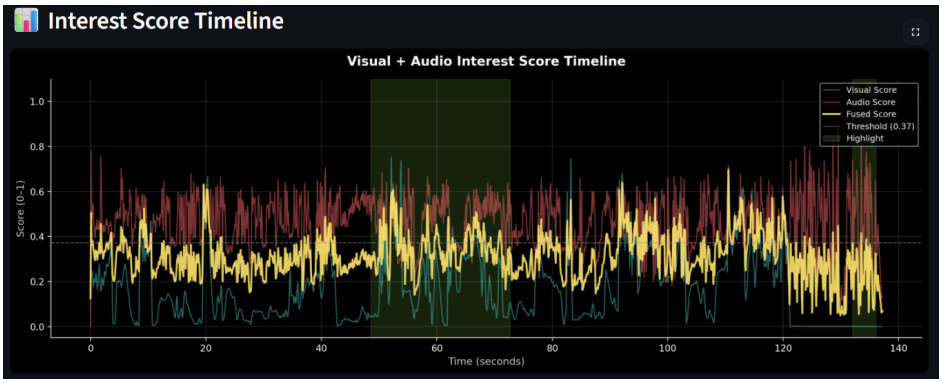


Figura 3: Graficul scorurilor de interes vizual şi audio pe axa timpului

2. Lista segmentelor detectate - fiecare segment este afişat ca un card cu timpul de start, timpul de final şi durata. Sunt prezentate şi metrici sumarizatoare: numărul total de segmente, durata cumulată a highlight-urilor şi procentajul de reducere a conţinutului.

3. Previzualizarea şi descărcarea fişierului final - montajul generat poate fi vizualizat direct în browser şi descărcat printr-un buton dedicat. Tranziţiile dintre segmente sunt de tip fade-in/fade-out, iar fişierul este deja optimizat pentru distribuţie.

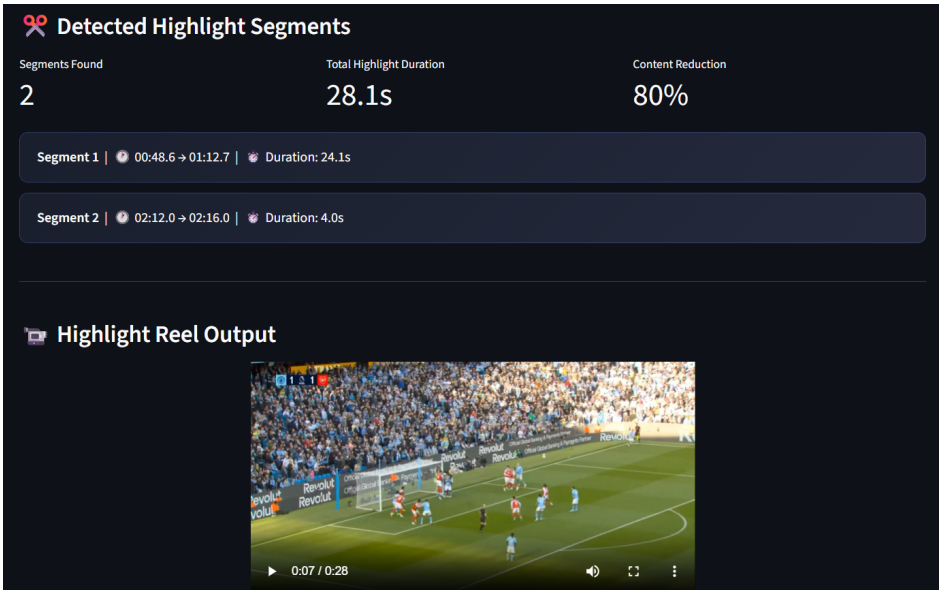


Figura 4: Segmentele detectate, metricile de compresie şi rezultatul final

5 Concluzii

Proiectul „Smart Video Highlight & GIF Generator" a demonstrat că extragerea inteligentă a momentelor cheie din materiale multimedia poate fi realizată eficient pe hardware accesibil, fără a necesita procesare în cloud, GPU dedicat sau modele masive de Deep Learning.

Obiectivele propuse în secţiunea 1.2 au fost îndeplinite integral:

- **Procesarea 100% locală** a fost realizată - toate operaţiunile se desfăşoară pe calculatorul utilizatorului, fără transmiterea datelor către servere externe.
- **Funcţionarea fără GPU** a fost demonstrată - algoritmii de flux optic Farnebäck, corelaţia histogramelor HSV şi extragerea descriptorilor audio librosa funcţionează exclusiv pe CPU.
- **Compatibilitatea cu hardware modest** a fost validată prin dezvoltarea şi testarea integrală pe un laptop cu 8 GB de RAM, graţie sistemului de chunked processing şi a optimizărilor de memorie descrise în secţiunea 3.5.
- **Controlul granular** este asigurat prin slidere intuitive pentru ponderile vizual/audio şi pentru percentila de prag, permiţând utilizatorului să adapteze algoritmul la tipul de conţinut procesat.
- **Exportul configurabil** pentru social media funcţionează corect atât în format landscape 16:9 cât şi în format vertical 9:16 cu crop centrat.

Faţă de soluţiile comerciale analizate în secţiunea 2.1, aplicaţia prezintă avantajul unic al **transparenţei algoritmice**: graficul timeline permite utilizatorului să înţeleagă exact cum au fost selec-

tate momentele de interes, spre deosebire de instrumentele „black-box” care nu oferă explicaţii ale deciziilor lor.

Ca **limitări**, aplicaţia nu realizează recunoaştere semantică a conţinutului (nu „înţelege” ce se întâmplă pe ecran, ci detectează doar mişcarea şi sunetul). De asemenea, calitatea detecţiei depinde de calitatea pistei audio - un videoclip cu fundal muzical uniform va produce scoruri audio mai puţin diferenţiate.

Ca **direcţii viitoare**, se pot integra: recunoaşterea textului de pe ecran (OCR) pentru identificarea scorului unui meci direct din graficele afişate, identificarea automată a vorbitorului activ (Speaker Diarization) pentru podcasturi cu mai mulţi participanţi, şi exportul direct pe platformele de social media prin API-urile acestora.

În concluzie, proiectul răspunde unei nevoi reale a comunităţii de creatori de conţinut short-form video, oferind un instrument practic, transparent şi accesibil financiar, care reduce semnificativ timpul necesar procesului de editare.

Declaraţie privind utilizarea modelelor de limbaj (LLM)

În conformitate cu Regulamentul Universităţii Politehnica din Bucureşti, declar că am utilizat un model de limbaj de tip LLM (Large Language Model) în următoarele scopuri limitate:

- **Stilizarea CSS** a interfeţei Streamlit - generarea regulilor de stil vizual (culori, gradienţi, umbre) pentru componentele UI din fişierul `app.py`.
- **Configurarea mediului de execuţie** - asistare la crearea structurii de mediu virtual Python (*virtual environment*), a fişierului `requirements.txt` şi a scriptului de lansare `start_app.bat`.
- **Planificarea iniţială** - realizarea unui plan preliminar al structurii proiectului pentru a stabili arhitectura modulară şi fluxul de procesare.

Logica de bază a aplicaţiei, implementarea algoritmilor de procesare vizuală (flux optic, histograme HSV), a descriptorilor audio (RMS, centroid spectral, onset strength), a mecanismului de fuziune a scorurilor, a optimizărilor de memorie (chunked processing, fade uint8) şi a pipeline-ului complet de procesare reprezintă contribuţia mea individuală.

Bibliografie

[1] G. Bradski (2000), *The OpenCV Library*, Dr. Dobb's Journal of Software Tools.

[2] G. Farnebäck (2003), *Two-Frame Motion Estimation Based on Polynomial Expansion*, Proceedings of the 13th Scandinavian Conference on Image Analysis (SCIA), pp. 363–370.

[3] ITU-T Recommendation H.264 (2003), *Advanced Video Coding for Generic Audiovisual Services*, International Telecommunication Union.

[4] B. McFee, C. Raffel, D. Liang, D. P. W. Ellis, M. McVicar, E. Battenberg, O. Nieto (2015), *librosa: Audio and Music Signal Analysis in Python*, Proceedings of the 14th Python in Science Conference, pp. 18–24.

[5] Zulko et al. (2015), *MoviePy: Video editing with Python*. https://zulko.github.io/moviepy/ (Accesat în aprilie 2026).

[6] A. C. Müller, S. Guido (2016), *Introduction to Machine Learning with Python: A Guide for Data Scientists*, O'Reilly Media, ISBN: 978-1-449-36941-5.

[7] Cisco Systems (2020), *Cisco Annual Internet Report (2018–2023)*, White Paper, https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html (Accesat în aprilie 2026).

[8] Adobe Inc. (Accesat în aprilie 2026), *Auto Reframe — Premiere Pro User Guide*, https://helpx.adobe.com/premiere-pro/using/auto-reframe.html.

[9] OpusClip (Accesat în aprilie 2026), *AI Video Repurposing Platform*, https://www.opus.pro/.

[10] CapCut — ByteDance (Accesat în aprilie 2026), *CapCut Video Editor*, https://www.capcut.com/.

[11] Gling (Accesat în aprilie 2026), *AI Video Editing for Content Creators*, https://www.gling.ai/.

[12] Streamlit Inc. (Accesat în aprilie 2026), *Streamlit — A faster way to build and share data apps*, https://docs.streamlit.io/.